- AirAd — Digital Ocean Deployment Requirements
  - Table of Contents
  - 1. Project Overview
    - 1.1 What is AirAd?
    - 1.2 Tech Stack Summary
    - 1.3 Application Components (6 Services)
    - 1.4 Backend Scale
  - 2. Architecture Diagram
  - 3. Server Requirements (Droplets)
    - 3.1 Recommended Setup — Single Droplet (Initial Production)
    - 3.2 Alternative — Managed Services (Recommended for Production)
    - 3.3 Minimum Viable (Budget Option — Not Recommended for Production)
  - 4. Network & Infrastructure
    - 4.1 Load Balancer
    - 4.2 Firewall Rules (DO Cloud Firewall)
    - 4.3 Ports Summary
    - 4.4 VPC (Virtual Private Cloud)
  - 5. Database Setup
    - 5.1 Database Engine
    - 5.2 Option A — Digital Ocean Managed Database (RECOMMENDED)
    - 5.3 Option B — Self-Hosted in Docker (Single Droplet)
  - 6. Redis Setup
    - 6.1 What Redis is Used For
    - 6.2 Option A — Digital Ocean Managed Redis (RECOMMENDED)
    - 6.3 Option B — Self-Hosted in Docker
  - 7. Storage Requirements (Spaces)
    - 7.1 Digital Ocean Spaces (S3-Compatible)
    - 7.2 Spaces Configuration
    - 7.3 Folder Structure in Spaces
  - 8. Environment Variables — Complete Reference
    - 8.1 Backend Environment Variables
      - Django Core
      - Database
      - Redis
      - Storage (DO Spaces / S3)
      - Stripe (Payments)
      - Twilio (SMS / OTP)
      - Firebase (Push Notifications)
      - Google Places API
      - Monitoring
      - JWT
      - SSL
      - Docker Compose Specific (if self-hosting DB)
    - 8.2 Frontend Environment Variables
    - 8.3 Total Sensitive Secrets Count: 17
  - 9. Services & Third-Party Dependencies
    - 9.1 Third-Party Service Accounts Required
    - 9.2 Stripe Webhook Setup
    - 9.3 Background Workers (Celery)
      - Celery Worker
      - Celery Beat (Scheduler)
    - 9.4 Caching Layer
  - 10. Deployment Process — Step by Step
    - 10.1 Pre-Deployment Setup (One-Time)
    - 10.2 Droplet Setup
    - 10.3 Deployment Order

    - 10.4 Docker Compose Deployment (Recommended Method)
    - 10.5 Frontend Build & Deployment
    - 10.6 Database Migrations
    - 10.7 Static Files
  - 11. SSL/TLS & Domain Configuration
    - 11.1 Domain Setup
    - 11.2 SSL Certificate
    - 11.3 Nginx SSL Notes
  - 12. Monitoring & Logging
    - 12.1 Application Monitoring
    - 12.2 Log Access
    - 12.3 Health Check Endpoints
  - 13. Backup Strategy
    - 13.1 Database Backups
    - 13.2 File Storage Backups
    - 13.3 Application Backups
  - 14. Scaling Considerations
    - 14.1 Initial Setup (Handles ~1,000 DAU)
    - 14.2 Vertical Scaling (Quick, ~5,000 DAU)
    - 14.3 Horizontal Scaling (Complex, ~50,000+ DAU)
    - 14.4 Database Scaling Notes
  - 15. Estimated Monthly Cost
    - 15.1 Recommended Setup (Managed Services)
    - 15.2 Budget Setup (Self-Hosted Everything)
    - 15.3 Minimum Viable (Staging/Demo Only)
    - 15.4 Third-Party Service Costs (Not Digital Ocean)
  - 16. Pre-Deployment Checklist
    - Infrastructure
    - Application
    - Database
    - Third-Party Services
    - Security
    - Verification
  - Appendix A: Key File Paths in Repository
  - Appendix B: Quick Reference Commands

AirAd — Digital Ocean Deployment Requirements

Document Version: 1.0
Date: February 26, 2026
Project: AirAd — Hyperlocal Vendor Discovery Platform
Purpose: Complete deployment specification for Digital Ocean. This document contains everything a DevOps engineer needs to deploy from zero — no assumptions, no ambiguity.

Table of Contents

1. Project Overview
2. Architecture Diagram
3. Server Requirements (Droplets)
4. Network & Infrastructure
5. Database Setup
6. Redis Setup
7. Storage Requirements (Spaces)
8. Environment Variables — Complete Reference

9. Services & Third-Party Dependencies
10. Deployment Process — Step by Step
11. SSL/TLS & Domain Configuration
12. Monitoring & Logging
13. Backup Strategy
14. Scaling Considerations
15. Estimated Monthly Cost
16. Pre-Deployment Checklist

---

1. Project Overview

1.1 What is AirAd?

AirAd is a hyperlocal vendor discovery platform that connects nearby customers with local vendors (restaurants, shops, services) using geo-based search, AR-ready vendor ranking, voice search, subscription tiers, and real-time discounts.

1.2 Tech Stack Summary

Layer	Technology	Version
Backend Framework	Django + Django REST Framework	5.1.5 + 3.15.2
Backend Language	Python	3.12
Database	PostgreSQL + PostGIS	16 + 3.4
Cache & Message Broker	Redis	7.x
Task Queue	Celery + Celery Beat	5.4.0
WSGI Server	Gunicorn	23.0.0
Frontend Framework	React + TypeScript	18.3.1 + 5.7.2
Frontend Build Tool	Vite	7.3.1
Reverse Proxy	Nginx	1.25.x
Containerization	Docker + Docker Compose	Latest stable
API Documentation	drf-spectacular (OpenAPI/Swagger)	0.27.2

1.3 Application Components (6 Services)

#	Service	Description	Port
1	**PostgreSQL + PostGIS**	Primary database with geospatial extensions	5432
2	**Redis**	Cache layer + Celery broker + Celery result backend	6379
3	**Django Backend (Gunicorn)**	REST API server — 4 Gunicorn workers	8000 (internal)
4	**Celery Worker**	Background task processor — concurrency=4	N/A (internal)
5	**Celery Beat**	Periodic task scheduler — **EXACTLY 1 replica always**	N/A (internal)
6	**Nginx**	Reverse proxy — only public-facing service	80, 443

The React frontend is a **static SPA** (Single Page Application) — built with `npm run build` → outputs to `dist/` folder → served by Nginx directly. No Node.js server needed in production.

1.4 Backend Scale

- **16 Django apps:** accounts, audit, geo, tags, vendors, imports, field_ops, qa, analytics, subscriptions, discovery, governance, vendor_portal, payments, reels, notifications
- **~98 REST API endpoints** across 3 phases (Admin Portal, Customer/Discovery, Vendor Portal)
- **18 Celery Beat scheduled tasks** (QA scans, tag expiry, discount scheduling, subscription checks, churn detection, flash deals, happy hours, voicebot freshness, monthly reports, activation checks, GDPR purge)

---

2. Architecture Diagram

```
                    ┌──────────────────────┐
                    │   Digital Ocean      │
                    │   Load Balancer      │
                    │   (SSL Termination)  │
                    │   :443 → :80         │
                    └──────────────────────┘
                              │
                              ▼
                    ┌──────────────────────┐
                    │       Nginx          │
                    │   (Reverse Proxy)    │
                    │   Port 80            │
                    └──────────────────────┘
                       │                 │
                       │                 │
              /api/*   │                 │   /* (all other)
                       │                 │
                       ▼                 ▼
          ┌──────────────────┐   ┌──────────────────┐
          │  Django/Gunicorn │   │  Static SPA Files │
          │  Backend         │   │  (React dist/)    │
          │  Port 8000       │   │  Served by Nginx  │
          └──────────────────┘   └──────────────────┘
                │        │
                ▼        ▼
        ┌──────────┐  ┌──────────┐
        │ PostgreSQL │ │   Redis   │
        │ + PostGIS  │ │ (Cache +  │
        │ Port 5432  │ │  Broker)  │
        └──────────┘  │ Port 6379 │
                      └──────────┘
                            │
                            ▼
                    ┌──────────────────────┐
                    │   Celery Worker      │
                    │   (4 concurrency)    │
                    ├──────────────────────┤
                    │   Celery Beat        │
                    │   (1 replica ONLY)   │
                    └──────────────────────┘

External Services:
  → Stripe (payments)
  → Twilio (SMS/OTP)
  → Firebase (push notifications)
  → Google Places API (vendor data import)
  → DO Spaces / S3 (file storage)
  → Sentry (error tracking)
```

---

3. Server Requirements (Droplets)

3.1 Recommended Setup — Single Droplet (Initial Production)

For launch/MVP, all services can run on a **single droplet using Docker Compose**. The existing `docker-compose.yml` is production-ready.

Resource	Specification
Droplet Plan	General Purpose — `g-4vcpu-16gb`
vCPUs	4 dedicated vCPUs
RAM	16 GB
Storage	100 GB NVMe SSD (minimum)
OS	Ubuntu 24.04 LTS (latest LTS)
Region	Choose closest to target users (e.g., `blr1` for India, `nyc1` for US)
Monthly Cost	~$96/month

Why 16 GB RAM?

- PostgreSQL + PostGIS: ~2-3 GB (geospatial queries are memory-intensive)
- Redis: ~512 MB
- Gunicorn (4 workers): ~1-2 GB
- Celery Worker (4 concurrency): ~1-2 GB
- Celery Beat: ~256 MB
- Nginx: ~128 MB
- Docker overhead: ~1 GB
- OS + buffers: ~2 GB
- **Total estimated: ~10 GB active, 6 GB headroom**

3.2 Alternative — Managed Services (Recommended for Production)

Instead of running PostgreSQL and Redis inside Docker on the droplet, use Digital Ocean Managed Databases. This gives automatic backups, failover, and monitoring.

Resource	Specification
App Droplet	General Purpose — `g-2vcpu-8gb` ($48/month)
Managed PostgreSQL	Basic — `db-s-2vcpu-4gb` ($60/month) — see §5
Managed Redis	Basic — `db-s-1vcpu-2gb` ($15/month) — see §6

This approach is **strongly recommended** because:

- Automated daily backups with point-in-time recovery
- Automatic failover for high availability
- No manual PostgreSQL/PostGIS maintenance
- Connection pooling built-in
- Monitoring and alerts included

3.3 Minimum Viable (Budget Option — Not Recommended for Production)

Resource	Specification
Droplet Plan	Basic — `s-2vcpu-4gb`
vCPUs	2 shared vCPUs
RAM	4 GB
Storage	80 GB SSD
Monthly Cost	~$24/month

⚠️ **Warning:** This is suitable only for staging/demo. Production traffic with PostGIS queries, Celery workers, and 4 Gunicorn workers will cause OOM kills on 4 GB RAM.

---

4. Network & Infrastructure

4.1 Load Balancer

Setting	Value
Required?	Yes — for SSL termination and future horizontal scaling
DO Product	Digital Ocean Load Balancer
Type	Regional
Algorithm	Round Robin
Forwarding Rules	HTTPS :443 → HTTP :80 (Nginx on droplet)
Health Check	HTTP :80, Path: `/nginx-health`, Interval: 10s, Timeout: 5s
Sticky Sessions	Not required (JWT-based auth, no server-side sessions for API)
Monthly Cost	$12/month

4.2 Firewall Rules (DO Cloud Firewall)

Rule	Type	Protocol	Port Range	Source
SSH	Inbound	TCP	22	Your office IP / VPN only
HTTP	Inbound	TCP	80	Load Balancer only (LB IP)
HTTPS	Inbound	TCP	443	Load Balancer only (LB IP)
PostgreSQL	Inbound	TCP	5432	Droplet private IP only (or Managed DB handles this)
Redis	Inbound	TCP	6379	Droplet private IP only (or Managed DB handles this)
All Outbound	Outbound	ALL	ALL	0.0.0.0/0 (needed for Stripe, Twilio, Google, Sentry, S3)

CRITICAL: Never expose ports 5432 (PostgreSQL) or 6379 (Redis) to the public internet.

4.3 Ports Summary

Port	Service	Exposed To
22	SSH	Office IP only
80	Nginx (HTTP)	Load Balancer only
443	Load Balancer (HTTPS)	Public internet
5432	PostgreSQL	Internal/Private network only
6379	Redis	Internal/Private network only
8000	Gunicorn	Internal Docker network only (NOT port-mapped to host)

4.4 VPC (Virtual Private Cloud)

Create a VPC for all resources in the same region:

- **Name:** `airaad-vpc`
- **IP Range:** `10.10.0.0/16` (default is fine)
- Place droplet, managed DB, managed Redis, and load balancer in this VPC
- All internal communication happens over private IPs — no public internet exposure

---

5. Database Setup

5.1 Database Engine

Setting	Value
Engine	PostgreSQL 16
Extension Required	PostGIS 3.4 (geospatial — **MANDATORY**, app will not start without it)
Django Engine	`django.contrib.gis.db.backends.postgis`

5.2 Option A — Digital Ocean Managed Database (RECOMMENDED)

Setting	Value
Plan	Basic — `db-s-2vcpu-4gb`
vCPUs	2
RAM	4 GB
Storage	38 GB (auto-expandable)
Monthly Cost	$60/month
Standby Nodes	0 (initial), add 1 for HA later ($60/month extra)

Post-creation steps (MUST DO):

1. Enable PostGIS extension on the database:

```
CREATE EXTENSION IF NOT EXISTS postgis;
```

2. Create the application database:

```
CREATE DATABASE airaad_db;
\c airaad_db
CREATE EXTENSION IF NOT EXISTS postgis;
```

3. Create application user (if not using default `doadmin`):

```
CREATE USER airaad WITH PASSWORD '<STRONG_PASSWORD_HERE>';
GRANT ALL PRIVILEGES ON DATABASE airaad_db TO airaad;
```

4. Note the connection string from DO dashboard — format:

```
postgis://airaad:<PASSWORD>@<PRIVATE_HOST>:25060/airaad_db?sslmode=require
```

⚠️ Managed DB uses port **25060**, not 5432.

Connection Pooling:

- DO Managed Database has **built-in connection pooling** (PgBouncer)
- Enable it in DO dashboard → Database → Connection Pools
- Create pool: `airaad-pool`, Mode: `Transaction`, Size: 25
- Use the pool connection string for `DATABASE_URL` in production

Backup Strategy (Managed):

- Automatic daily backups — retained for 7 days (included in plan)
- Point-in-time recovery available
- Manual backup before major deployments: DO Dashboard → Database → Backups → Create Backup

5.3 Option B — Self-Hosted in Docker (Single Droplet)

If using the Docker Compose approach, PostgreSQL runs as a container. Already configured in `docker-compose.yml`:

```
postgres:
  image: postgis/postgis:16-3.4-alpine
  volumes:
    - postgres_data:/var/lib/postgresql/data
  environment:
    POSTGRES_DB: airaad_db
    POSTGRES_USER: airaad
    POSTGRES_PASSWORD: <SET_IN_.ENV>
```

Backup Strategy (Self-Hosted):

- Set up a cron job for `pg_dump`:

  ```
  # Daily at 3 AM — dump to /backups/ and upload to DO Spaces
  0 3 * * * docker exec airaad-postgres-1 pg_dump -U airaad airaad_db | gzip >
  /backups/airaad_db_$(date +\%Y\%m\%d).sql.gz
  ```

- Retain 30 days of backups
- Upload to DO Spaces for off-server storage
- **Test restore monthly**

---

6. Redis Setup

6.1 What Redis is Used For

Purpose	Redis DB	Description
Django Cache	DB 0	API response caching, session data
Celery Broker	DB 0	Task message queue for background jobs
Celery Result Backend	DB 0	Store task results (success/failure)

6.2 Option A — Digital Ocean Managed Redis (RECOMMENDED)

Setting	Value
Plan	Basic — `db-s-1vcpu-2gb`

Setting	Value
vCPU	1
RAM	2 GB
Eviction Policy	`allkeys-lru`
Monthly Cost	$15/month

Connection string format: `rediss://default:<PASSWORD>@<PRIVATE_HOST>:25061`

⚠️ Managed Redis uses `rediss://` (with SSL) and port **25061**.

6.3 Option B — Self-Hosted in Docker

Already configured in `docker-compose.yml`:

```
redis:
  image: redis:7-alpine
  command: redis-server --appendonly yes
  volumes:
    - redis_data:/data
```

If self-hosted, add password protection:

```
command: redis-server --appendonly yes --requirepass <REDIS_PASSWORD>
```

---

7. Storage Requirements (Spaces)

7.1 Digital Ocean Spaces (S3-Compatible)

The backend uses `django-storages` with `boto3` for file storage. DO Spaces is S3-compatible, so it works out of the box — no code changes needed.

Setting	Value
Required?	Yes — for media uploads (vendor photos, reels, claim proofs, CSV imports)
DO Product	Spaces Object Storage
Region	Same as droplet region
Bucket Name	`airaad-production`
CDN	Yes — Enable Spaces CDN for static assets and media
Access	Private (presigned URLs for media, public-read for static assets)
Monthly Cost	$5/month (250 GB storage + 1 TB transfer included)

7.2 Spaces Configuration

1. **Create Space** in DO Dashboard:
   - Name: `airaad-production`
   - Region: Same as droplet
   - File Listing: Restricted
2. **Create Spaces Access Key:**
   - DO Dashboard → API → Spaces Keys → Generate New Key

- Save the **Key** and **Secret** — these map to `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY`

3. **CDN Setup:**
   - Enable CDN on the Space
   - Custom subdomain: `cdn.yourdomain.com` (optional)
   - CDN endpoint will be: `https://airaad-production.<REGION>.cdn.digitaloceanspaces.com`

4. **Backend Configuration:**

```
AWS_ACCESS_KEY_ID=<Spaces_Key>
AWS_SECRET_ACCESS_KEY=<Spaces_Secret>
AWS_STORAGE_BUCKET_NAME=airaad-production
AWS_S3_REGION_NAME=<your_region>  # e.g., nyc3, blr1
AWS_S3_ENDPOINT_URL=https://<REGION>.digitaloceanspaces.com
```

5. **IMPORTANT — Additional Setting Required:** Add `AWS_S3_ENDPOINT_URL` to `config/settings/production.py` to point to DO Spaces instead of AWS S3:

```
AWS_S3_ENDPOINT_URL = env("AWS_S3_ENDPOINT_URL", default="")
```

This tells `boto3` to use Digital Ocean Spaces instead of AWS S3.

7.3 Folder Structure in Spaces

```
airaad-production/
├── static/          # Django static files (CSS, JS, admin assets)
├── media/
│   ├── vendors/     # Vendor photos and logos
│   ├── reels/       # Vendor reel videos
│   ├── claims/      # Claim proof documents
│   └── imports/     # CSV import files
```

---

8. Environment Variables — Complete Reference

8.1 Backend Environment Variables

Every variable listed below **MUST** be set. No defaults are safe for production.

Django Core

Variable	Description	Example Value
`SECRET_KEY`	Django secret key — generate with: `python -c "from django.core.management.utils import get_random_secret_key; print(get_random_secret_key())"`	`django-insecure-abc`
`DEBUG`	Must be `False` in production	`False`
`ALLOWED_HOSTS`	Comma-separated list of valid hostnames	`yourdomain.com,api.`
`DJANGO_SETTINGS_MODULE`	Settings module path	`config.settings.pro`
`ENCRYPTION_KEY`	AES-256-GCM key, 32-byte base64. Generate: `python -c "import os, base64; print(base64.b64encode(os.urandom(32)).decode())"`	`aBcDeFgHiJkLmNoPqRs`

Variable	Description	Example Value
NUM_PROXIES	Number of reverse proxies in front of Django (Load Balancer + Nginx = 2)	2
CORS_ALLOWED_ORIGINS	Comma-separated frontend URLs	https://yourdomain.
FRONTEND_URL	Frontend URL for CORS and email links	https://yourdomain.
PORT	Gunicorn bind port (used in start.sh)	8000

Database

Variable	Description	Example Value	Sensitive?
DATABASE_URL	PostGIS connection string	postgis://airaad:<PASSWORD>@<PRIVATE_HOST>:25060/airaad_db?sslmode=require	🔴 YES

Redis

Variable	Description	Example Value	Sensitive?
REDIS_URL	Redis connection string	rediss://default:<PASSWORD>@<PRIVATE_HOST>:25061	🔴 YES
CELERY_BROKER_URL	Celery message broker (same as REDIS_URL)	rediss://default:<PASSWORD>@<PRIVATE_HOST>:25061	🔴 YES
CELERY_RESULT_BACKEND	Celery result storage (same as REDIS_URL)	rediss://default:<PASSWORD>@<PRIVATE_HOST>:25061	🔴 YES

Storage (DO Spaces / S3)

Variable	Description	Example Value	Sensitive?
AWS_ACCESS_KEY_ID	DO Spaces access key	DO00XXXXXXXXXXXX	🔴 YES
AWS_SECRET_ACCESS_KEY	DO Spaces secret key	abcdefghijklmnop...	🔴 YES
AWS_STORAGE_BUCKET_NAME	Spaces bucket name	airaad-production	No
AWS_S3_REGION_NAME	Spaces region	nyc3	No
AWS_S3_ENDPOINT_URL	DO Spaces endpoint (NOT AWS)	https://nyc3.digitaloceanspaces.com	No
AWS_S3_PRESIGNED_URL_EXPIRY	Presigned URL lifetime in seconds	3600	No

Stripe (Payments)

Variable	Description	Example Value	Sensitive?
`STRIPE_SECRET_KEY`	Stripe API secret key	`sk_live_...`	🔴 YES
`STRIPE_PUBLISHABLE_KEY`	Stripe publishable key (also used in frontend)	`pk_live_...`	No
`STRIPE_WEBHOOK_SECRET`	Stripe webhook signing secret	`whsec_...`	🔴 YES
`STRIPE_PRICE_GOLD`	Stripe Price ID for Gold tier	`price_1Abc...`	No
`STRIPE_PRICE_DIAMOND`	Stripe Price ID for Diamond tier	`price_1Def...`	No
`STRIPE_PRICE_PLATINUM`	Stripe Price ID for Platinum tier	`price_1Ghi...`	No
`STRIPE_SILVER_PRICE_ID`	Stripe Price ID for Silver tier (free)	`price_1Jkl...`	No

Twilio (SMS / OTP)

Variable	Description	Example Value	Sensitive?
`TWILIO_ACCOUNT_SID`	Twilio Account SID	`AC...`	🔴 YES
`TWILIO_AUTH_TOKEN`	Twilio Auth Token	`auth_token_here`	🔴 YES
`TWILIO_PHONE_NUMBER`	Twilio sender phone number	`+12025551234`	No

Firebase (Push Notifications)

Variable	Description	Example Value	Sensitive?
`FIREBASE_CREDENTIALS_JSON`	Firebase service account JSON (base64-encoded or file path)	`eyJ0eXBlIjoic2Vydm...`	🔴 YES

Google Places API

Variable	Description	Example Value	Sensitive?
`GOOGLE_PLACES_API_KEY`	Google Cloud API key with Places API enabled	`AIzaSy...`	🔴 YES

Monitoring

Variable	Description	Example Value	Sensitive?
`SENTRY_DSN`	Sentry error tracking DSN	`https://abc123@o123.ingest.sentry.io/456`	🔴 YES

JWT

Variable	Description	Example Value	Sensitive?
`JWT_ACCESS_LIFETIME_MINUTES`	Access token lifetime	`60`	No
`JWT_REFRESH_LIFETIME_DAYS`	Refresh token lifetime	`7`	No

SSL

Variable	Description	Example Value	Sensitive?
`SECURE_SSL_REDIRECT`	Set to `False` if SSL terminates at Load Balancer	`False`	No

Docker Compose Specific (if self-hosting DB)

Variable	Description	Example Value	Sensitive?
`POSTGRES_DB`	Database name	`airaad_db`	No
`POSTGRES_USER`	Database user	`airaad`	No
`POSTGRES_PASSWORD`	Database password	`<STRONG_PASSWORD>`	🔴 YES

8.2 Frontend Environment Variables

The frontend is a Vite React SPA. Environment variables must be prefixed with `VITE_`.

Variable	Description	Example Value	Sensitive?
`VITE_API_BASE_URL`	Backend API URL (must be reachable from browser)	`https://api.yourdomain.com`	No
`VITE_APP_ENV`	Environment identifier	`production`	No

⚠️ Frontend env vars are **baked into the build at compile time** (`npm run build`). They are NOT set at runtime. You must rebuild the frontend whenever these change.

8.3 Total Sensitive Secrets Count: 17

All 🔴 marked variables above must be stored securely. Never commit to git. Use DO App Platform environment variables, or store in a `.env` file with `chmod 600` on the droplet.

---

9. Services & Third-Party Dependencies

9.1 Third-Party Service Accounts Required

Service	Purpose	Account Setup URL	What to Configure
Stripe	Subscription payments (Gold/Diamond/Platinum tiers)	https://dashboard.stripe.com	Create 4 Products + Prices, set up Webhook endpoin `https://api.yourdomain.com/api/v1/paym`
Twilio	OTP SMS for vendor phone verification and customer auth	https://console.twilio.com	Purchase a phone number, note SID + Auth Token
Firebase	Push notifications to mobile app	https://console.firebase.google.com	Create project, download service account JSON, bas
Google Cloud	Places API for vendor data import	https://console.cloud.google.com	Enable Places API (New), create API key, restrict to se
Sentry	Error tracking and performance monitoring	https://sentry.io	Create Django project, get DSN
Slack (optional)	Deploy notifications	https://api.slack.com/apps	Create incoming webhook URL

9.2 Stripe Webhook Setup

After deployment, configure Stripe webhook:

1. Go to Stripe Dashboard → Developers → Webhooks → Add endpoint

2. **Endpoint URL:** `https://api.yourdomain.com/api/v1/payments/webhook/`
3. **Events to listen for:**
   - `checkout.session.completed`
   - `invoice.paid`
   - `invoice.payment_failed`
   - `customer.subscription.updated`
   - `customer.subscription.deleted`
4. Copy the **Signing Secret** (`whsec_...`) → set as `STRIPE_WEBHOOK_SECRET`

9.3 Background Workers (Celery)

Celery Worker

- **Command:** `celery -A celery_app worker --loglevel=info --concurrency=4`
- Processes all background tasks: imports, notifications, analytics, payment sync
- Can scale concurrency based on load

Celery Beat (Scheduler)

- **Command:** `celery -A celery_app beat --loglevel=info --scheduler django_celery_beat.schedulers:DatabaseScheduler`
- ⚠️ **CRITICAL: Must run EXACTLY 1 instance.** Running multiple Beat instances will cause duplicate task execution.
- Manages 18 periodic tasks:

Task	Schedule	Purpose
`weekly_gps_drift_scan`	Sunday 02:00 UTC	QA: detect GPS coordinate drift
`daily_duplicate_scan`	Daily 03:00 UTC	QA: find duplicate vendors
`expire_promotion_tags`	Every 5 minutes	Expire time-limited promotion tags
`expire_temporary_suspensions`	Daily 01:00 UTC	Lift expired vendor suspensions
`purge_deleted_user_data`	Daily 02:00 UTC	GDPR: purge anonymized accounts >30 days
`purge_old_analytics_events`	Daily 03:30 UTC	Delete analytics events >90 days
`deprecate_unused_tags`	1st of month 04:00 UTC	Flag low-usage tags
`audit_log_retention_check`	1st of month 05:00 UTC	Warn about logs approaching 1-year
`generate_time_context_tags`	Every hour	Auto-generate time-based tags
`discount_scheduler`	Every 1 minute	Activate/deactivate scheduled discounts
`subscription_expiry_check`	Daily midnight UTC	Check expired subscriptions
`hourly_tag_assignment`	Every hour	Auto-assign tags to vendors
`flash_deal_trigger`	Every 5 minutes	Trigger flash deals for Platinum vendors
`auto_happy_hour_trigger`	Every 15 minutes	Auto happy hour for Platinum vendors
`voicebot_freshness_check`	Daily 06:00 UTC	Check voice bot data freshness
`vendor_churn_check`	Daily 07:00 UTC	Detect churning vendors (14-day inactivity)
`vendor_monthly_report`	1st of month 06:00 UTC	Generate monthly vendor reports
`vendor_activation_check`	Daily 02:00 UTC	Progress vendor activation stages

9.4 Caching Layer

Redis is used for Django caching with `django.core.cache.backends.redis.RedisCache`. The `CONN_MAX_AGE=60` on the database keeps PostgreSQL connections alive for 60 seconds for performance.

---

10. Deployment Process — Step by Step

10.1 Pre-Deployment Setup (One-Time)

```
STEP 1: Create Digital Ocean resources
  ├── 1a. Create VPC: airaad-vpc
  ├── 1b. Create Droplet (g-4vcpu-16gb or g-2vcpu-8gb) in VPC
  ├── 1c. Create Managed PostgreSQL in same VPC (if using managed)
  ├── 1d. Create Managed Redis in same VPC (if using managed)
  ├── 1e. Create Spaces bucket: airaad-production
  ├── 1f. Create Load Balancer in same VPC
  └── 1g. Create Cloud Firewall and attach to droplet

STEP 2: Configure DNS
  ├── 2a. Point yourdomain.com → Load Balancer IP
  ├── 2b. Point api.yourdomain.com → Load Balancer IP (if using subdomain)
  └── 2c. Configure SSL certificate on Load Balancer (Let's Encrypt auto)

STEP 3: Set up PostGIS extension on database
  └── Connect to managed DB and run: CREATE EXTENSION IF NOT EXISTS postgis;

STEP 4: Generate all Spaces access keys
  └── DO Dashboard → API → Spaces Keys → Generate
```

10.2 Droplet Setup

```
# SSH into the droplet
ssh root@<DROPLET_IP>

# 1. Update system
apt update && apt upgrade -y

# 2. Install Docker
curl -fsSL https://get.docker.com -o get-docker.sh
sh get-docker.sh

# 3. Install Docker Compose
apt install -y docker-compose-plugin

# 4. Create app user (do NOT run as root)
adduser airaad
usermod -aG docker airaad
su - airaad

# 5. Clone repository
git clone <REPO_URL> /home/airaad/app
cd /home/airaad/app/airaad

# 6. Create .env file
cp .env.example .env
nano .env   # Fill in ALL variables from Section 8

# 7. Set secure permissions
chmod 600 .env
```

10.3 Deployment Order

The order matters. Follow this sequence exactly:

```
PHASE 1: Infrastructure (must be up first)
  ├── 1. PostgreSQL — must be running + PostGIS enabled + database created
```

```
  └── 2. Redis — must be running and accessible

PHASE 2: Backend
  ├── 3. Build backend Docker image
  ├── 4. Run database migrations: python manage.py migrate --noinput
  ├── 5. Create superuser: python manage.py createsuperuser
  ├── 6. Collect static files: python manage.py collectstatic --noinput
  ├── 7. Start Gunicorn (backend container)
  ├── 8. Start Celery Worker
  └── 9. Start Celery Beat (EXACTLY 1 instance)

PHASE 3: Frontend
  ├── 10. Build frontend: npm run build (with correct VITE_API_BASE_URL)
  └── 11. Copy dist/ to Nginx serving directory

PHASE 4: Reverse Proxy
  └── 12. Start Nginx with production config
```

10.4 Docker Compose Deployment (Recommended Method)

```bash
cd /home/airaad/app/airaad

# Build all images
docker compose build

# Start database and Redis first, wait for health checks
docker compose up -d postgres redis
sleep 15  # Wait for DB to be ready

# Run migrations
docker compose run --rm backend python manage.py migrate --noinput

# Create initial superuser (interactive)
docker compose run --rm backend python manage.py createsuperuser

# Collect static files
docker compose run --rm backend python manage.py collectstatic --noinput

# Start all services
docker compose up -d

# Verify all containers are running
docker compose ps

# Check logs
docker compose logs -f backend
docker compose logs -f celery-worker
docker compose logs -f celery-beat
```

If using Managed Database + Managed Redis:
Remove `postgres` and `redis` services from `docker-compose.yml` and update environment variables to point to managed service URLs.

10.5 Frontend Build & Deployment

The frontend needs to be built before deployment. There is **no frontend Dockerfile in the repo** — you need to create one:

```dockerfile
# airaad/frontend/Dockerfile
FROM node:20-alpine AS builder
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
ARG VITE_API_BASE_URL
ARG VITE_APP_ENV=production
ENV VITE_API_BASE_URL=$VITE_API_BASE_URL
ENV VITE_APP_ENV=$VITE_APP_ENV
RUN npm run build

FROM nginx:1.25-alpine
COPY --from=builder /app/dist /usr/share/nginx/html
```

```
COPY <<'EOF' /etc/nginx/conf.d/default.conf
server {
    listen 80;
    root /usr/share/nginx/html;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }

    location /assets/ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }
}
EOF
EXPOSE 80
```

Build with:

```
docker build \
  --build-arg VITE_API_BASE_URL=https://api.yourdomain.com \
  --build-arg VITE_APP_ENV=production \
  -t airaad-frontend:latest \
  ./airaad/frontend
```

10.6 Database Migrations

Migrations are handled automatically by `start.sh` on container startup, or run manually:

```
# Run all pending migrations
docker compose run --rm backend python manage.py migrate --noinput

# Check for missing migrations (should show "No changes detected")
docker compose run --rm backend python manage.py makemigrations --check --dry-run

# Show migration status
docker compose run --rm backend python manage.py showmigrations
```

10.7 Static Files

Static files are served from either:

- **DO Spaces** (recommended): `collectstatic` uploads to S3/Spaces automatically when `AWS_STORAGE_BUCKET_NAME` is set
- **Local filesystem**: `collectstatic` copies to `staticfiles/` dir, served by Nginx

```
docker compose run --rm backend python manage.py collectstatic --noinput
```

---

11. SSL/TLS & Domain Configuration

11.1 Domain Setup

Record	Type	Name	Value
Root domain	A	`@`	Load Balancer IP
www subdomain	CNAME	`www`	`yourdomain.com`
API subdomain (optional)	A	`api`	Load Balancer IP

Record	Type	Name	Value
CDN (if custom)	CNAME	`cdn`	`airaad-production.<region>.cdn.digitaloceanspaces.com`

11.2 SSL Certificate

Setting	Value
Method	Let's Encrypt via DO Load Balancer (automatic, free)
Setup	DO Dashboard → Load Balancer → Settings → SSL → Add Certificate → Let's Encrypt
Domains	`yourdomain.com`, `www.yourdomain.com`, `api.yourdomain.com`
Renewal	Automatic (managed by Digital Ocean)
Backend SSL	Not required — Load Balancer terminates SSL, forwards HTTP to Nginx

11.3 Nginx SSL Notes

Since the DO Load Balancer handles SSL termination:

- Nginx listens on port **80 only** (HTTP)
- The `SECURE_PROXY_SSL_HEADER` setting in Django trusts `X-Forwarded-Proto: https` from the Load Balancer
- `SECURE_SSL_REDIRECT = False` to prevent redirect loops (Load Balancer handles HTTPS redirect)
- HSTS headers are set by Django (`SECURE_HSTS_SECONDS = 31536000`)

---

12. Monitoring & Logging

12.1 Application Monitoring

Tool	Purpose	Setup
Sentry	Error tracking, performance monitoring	Set `SENTRY_DSN` env var
DO Monitoring	Droplet CPU/RAM/Disk/Network metrics	Enable in DO Dashboard (free)
DO Database Insights	Query performance, connections, replication lag	Built into Managed DB (free)

12.2 Log Access

```
# All service logs
docker compose logs -f

# Backend only
docker compose logs -f backend

# Celery Worker
docker compose logs -f celery-worker

# Celery Beat
docker compose logs -f celery-beat

# Nginx access logs
docker compose exec nginx cat /var/log/nginx/access.log
```

Production logging is **JSON-formatted** for easy parsing by log aggregation tools.

12.3 Health Check Endpoints

Endpoint	Expected Response	Purpose
`GET /nginx-health`	HTTP 200 `"healthy\n"`	Nginx is alive (used by Load Balancer)
`GET /api/v1/health/`	HTTP 200 JSON	Django app + DB is healthy

---

13. Backup Strategy

13.1 Database Backups

Method	Frequency	Retention	Tested?
Managed DB auto-backup	Daily	7 days (included)	✅ DO manages
Manual pre-deploy backup	Before each deployment	Until next successful deploy	Manual trigger
Monthly full export	1st of month	90 days on DO Spaces	Cron job

13.2 File Storage Backups

- DO Spaces data is **not automatically backed up** by Digital Ocean
- Enable **Spaces versioning** for protection against accidental deletion
- For critical data, set up cross-region replication or scheduled `s3cmd sync` to a second bucket

13.3 Application Backups

- **Code:** Git repository is the backup (GitHub/GitLab)
- **Environment variables:** Keep a secure copy of `.env` in a password manager (1Password, Bitwarden)
- **Docker images:** Tag and push to a container registry (DO Container Registry or Docker Hub)

---

14. Scaling Considerations

14.1 Initial Setup (Handles ~1,000 DAU)

The single-droplet Docker Compose setup with Managed DB/Redis handles:

- ~1,000 daily active users
- ~50 concurrent API requests
- ~500 vendors in database
- ~10 Celery tasks per minute

14.2 Vertical Scaling (Quick, ~5,000 DAU)

Action	When	Cost Delta
Resize droplet to `g-8vcpu-32gb`	CPU consistently >70%	+$96/month
Increase Gunicorn workers to 8	More API concurrency needed	$0 (config change)
Increase Celery concurrency to 8	Task queue backing up	$0 (config change)
Upgrade Managed DB to `db-s-4vcpu-8gb`	Slow queries, high connections	+$60/month
Upgrade Managed Redis to `db-s-2vcpu-4gb`	Cache evictions increasing	+$15/month

14.3 Horizontal Scaling (Complex, ~50,000+ DAU)

Action	When	What to Do
Separate backend droplets	Single droplet maxed out	Run 2-3 backend droplets behind Load Balancer
Separate Celery worker droplet	Background tasks competing with API for CPU	Dedicate a droplet to Celery Worker + Beat
Read replicas for PostgreSQL	DB read queries are bottleneck	Add standby node on Managed DB ($60/month)
Redis cluster	Cache size exceeds single node	Upgrade to high-availability Redis
CDN for frontend	Global audience, slow initial loads	Serve frontend from Cloudflare/CDN, not droplet
DO App Platform	Want managed container orchestration	Migrate from Docker Compose to App Platform

14.4 Database Scaling Notes

- PostGIS spatial queries (ST_DWithin, distance calculations) are CPU and memory intensive
- Index all `location` (geography) columns with `GIST` indexes (already handled by Django migrations)
- If vendor count exceeds 100K, consider spatial partitioning
- `CONN_MAX_AGE=60` is already set — avoids connection creation overhead

---

15. Estimated Monthly Cost

15.1 Recommended Setup (Managed Services)

Resource	Plan	Monthly Cost
Droplet (App Server)	General Purpose `g-2vcpu-8gb`	$48.00
Managed PostgreSQL	Basic `db-s-2vcpu-4gb`	$60.00
Managed Redis	Basic `db-s-1vcpu-2gb`	$15.00
Load Balancer	Regional	$12.00
Spaces (Storage + CDN)	250 GB included	$5.00
DO Monitoring	Basic	Free
Bandwidth	8 TB included with droplet	Free (included)
Backups	Droplet weekly snapshots	~$9.60 (20% of droplet)
TOTAL		**$149.60/month**

15.2 Budget Setup (Self-Hosted Everything)

Resource	Plan	Monthly Cost
Droplet (Everything on one)	General Purpose `g-4vcpu-16gb`	$96.00
Load Balancer	Regional	$12.00
Spaces (Storage + CDN)	250 GB included	$5.00
Backups	Droplet weekly snapshots	~$19.20 (20% of droplet)

Resource	Plan	Monthly Cost
TOTAL		**$132.20/month**

15.3 Minimum Viable (Staging/Demo Only)

Resource	Plan	Monthly Cost
Droplet (Everything)	Basic `s-2vcpu-4gb`	$24.00
Spaces	250 GB included	$5.00
TOTAL		**$29.00/month**

15.4 Third-Party Service Costs (Not Digital Ocean)

Service	Free Tier	Paid Estimate
Stripe	Free (2.9% + $0.30 per transaction)	Transaction-based
Twilio	Trial included	$\sim 1/month$+0.0079/SMS
Firebase	Free tier (10K notifications/month)	Free for MVP scale
Google Places API	$200/month free credit	Usually free for MVP
Sentry	Free tier (5K errors/month)	Free for MVP scale
GitHub	Free for private repos	Free

---

16. Pre-Deployment Checklist

Infrastructure

- ☐ VPC created in target region
- ☐ Droplet provisioned and SSH accessible
- ☐ Managed PostgreSQL created with PostGIS extension enabled
- ☐ Managed Redis created
- ☐ DO Spaces bucket created with access keys
- ☐ Load Balancer created with SSL certificate
- ☐ Cloud Firewall configured and attached to droplet
- ☐ DNS records pointing to Load Balancer IP

Application

- ☐ Docker and Docker Compose installed on droplet
- ☐ Repository cloned to droplet
- ☐ `.env` file created with ALL variables from Section 8
- ☐ `.env` file has `chmod 600` permissions
- ☐ Frontend Dockerfile created (see Section 10.5)
- ☐ `AWS_S3_ENDPOINT_URL` added to production settings (for DO Spaces)
- ☐ `NUM_PROXIES=2` set (Load Balancer + Nginx)

Database

- ☐ PostGIS extension created on the database
- ☐ `DATABASE_URL` verified with `psql` connection test

- ☐ Migrations applied successfully: `python manage.py migrate`
- ☐ Superuser created: `python manage.py createsuperuser`
- ☐ `python manage.py check --deploy` returns 0 issues

Third-Party Services

- ☐ Stripe account created, products/prices configured
- ☐ Stripe webhook endpoint configured with correct URL
- ☐ Twilio account created, phone number purchased
- ☐ Firebase project created, service account JSON ready
- ☐ Google Places API key created and restricted to server IP
- ☐ Sentry project created, DSN obtained

Security

- ☐ `DEBUG=False` verified
- ☐ `SECRET_KEY` is unique, random, 50+ characters
- ☐ `ENCRYPTION_KEY` is properly generated (32 bytes, base64)
- ☐ All `ALLOWED_HOSTS` entries are correct (no wildcards)
- ☐ `CORS_ALLOWED_ORIGINS` lists only production frontend URLs
- ☐ SSH key-based authentication only (password auth disabled)
- ☐ Root login disabled on droplet
- ☐ Firewall rules verified — no unnecessary ports exposed
- ☐ PostgreSQL and Redis NOT accessible from public internet

Verification

- ☐ `GET /nginx-health` returns 200
- ☐ `GET /api/v1/health/` returns 200
- ☐ `GET /api/v1/docs/` loads Swagger UI
- ☐ Frontend loads at root URL
- ☐ API returns 401 for unauthenticated requests
- ☐ Login flow works (obtain JWT token)
- ☐ Celery Worker is processing tasks (check logs)
- ☐ Celery Beat is scheduling tasks (check logs)
- ☐ File upload to Spaces works
- ☐ Stripe checkout flow works (test mode first)

Appendix A: Key File Paths in Repository

```
AirAds-web/
├── airaad/
│   ├── backend/
│   │   ├── apps/                   # 16 Django applications
│   │   ├── config/
│   │   │   ├── settings/
│   │   │   │   ├── base.py         # Shared settings
│   │   │   │   ├── production.py   # Production overrides
│   │   │   │   ├── development.py  # Dev overrides
│   │   │   │   └── test.py         # Test overrides
│   │   │   ├── urls.py             # Root URL configuration (~98 endpoints)
│   │   │   ├── wsgi.py             # WSGI entry point
│   │   │   └── asgi.py             # ASGI entry point
│   │   ├── core/                   # Shared utilities, middleware, exceptions
│   │   ├── requirements/
│   │   │   ├── base.txt            # Shared dependencies
│   │   │   ├── production.txt      # Production (gunicorn, psycopg2)
│   │   │   ├── development.txt     # Dev tools
│   │   │   └── test.txt            # Test dependencies
```

```
│   │   ├── celery_app.py          # Celery config + 18 Beat schedules
│   │   ├── Dockerfile             # Multi-stage (builder → dev → production)
│   │   ├── start.sh               # Container entrypoint script
│   │   ├── manage.py              # Django management
│   │   └── .env.example           # Backend env var template
│   ├── frontend/
│   │   ├── src/                   # React TypeScript source
│   │   ├── package.json           # Node.js dependencies
│   │   ├── vite.config.ts         # Vite build configuration
│   │   └── .env.example           # Frontend env var template
│   ├── nginx/
│   │   ├── nginx.conf             # Production Nginx config
│   │   └── nginx.dev.conf         # Development Nginx config
│   ├── docker-compose.yml         # Production Docker Compose (6 services)
│   └── .env.example               # Root env var template
├── .github/workflows/
│   ├── ci.yml                     # CI pipeline (lint, test, security, build)
│   ├── deploy-production.yml      # Production deploy workflow
│   ├── deploy-development.yml     # Development deploy workflow
│   └── deploy-staging.yml         # Staging deploy workflow
└── docs/
    └── DEPLOYMENT_REQUIREMENTS.md # This file
```

Appendix B: Quick Reference Commands

```
# Start all services
docker compose up -d

# Stop all services
docker compose down

# Rebuild and restart
docker compose up -d --build

# View running containers
docker compose ps

# Run Django management commands
docker compose run --rm backend python manage.py <command>

# View logs (follow mode)
docker compose logs -f <service_name>

# Enter a running container
docker compose exec backend bash

# Restart a single service
docker compose restart <service_name>

# Check disk usage
docker system df

# Clean up unused images
docker system prune -f

# Database shell
docker compose exec backend python manage.py dbshell

# Django system check for deployment
docker compose run --rm backend python manage.py check --deploy
```

END OF DOCUMENT

This document was generated from deep analysis of the AirAd codebase. All specifications are based on actual code configuration — not assumptions. Hand this to your DevOps engineer and they should be able to deploy without any additional questions.